

EXPLAINABLE AI CODE GENERATION ASSISTANT

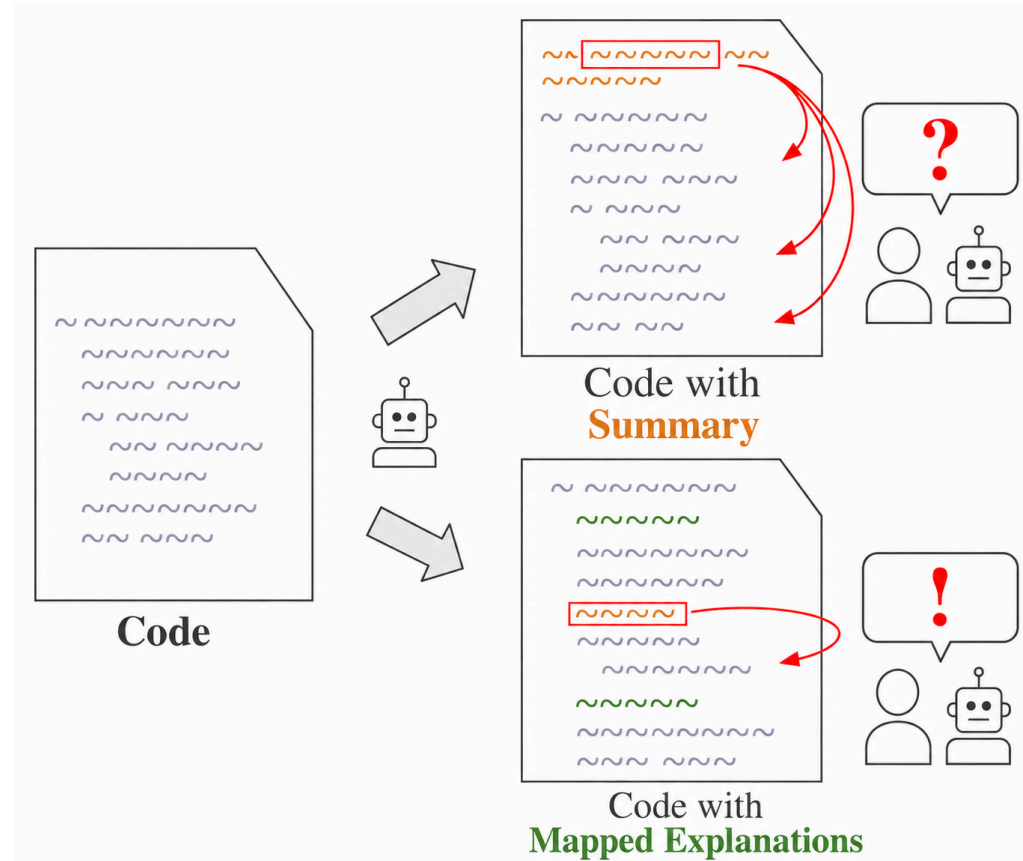
Author: Iuliia Liutova

Supervisor: Alexander Avdiushenko

Neapolis University Paphos

PROBLEM

- LLMs write code, but do not explain decisions clearly
- No mapping between summary and code



GOAL

Build a code assistant that **generates code and explanations** aligned with the code.

☒ Structured Granularity ☐

The function filters a list of users to find those who are currently active and have been members for more than a year. It then formats each qualifying user's name and email into a string. The result is a list of these strings.

```
1 def process_user_data(users):
2     active_users = [u for u in users if u['status'] == 'active']
3
4     from datetime import datetime, timedelta
5     one_year_ago = datetime.now() - timedelta(days=365)
6     long_term_users = [
7         u for u in active_users if u['join_date'] < one_year_ago
8     ]
9
10    results = []
11    for u in long_term_users:
12        results.append(f"{u['name']} <{u['email']}>")
13    return results
```

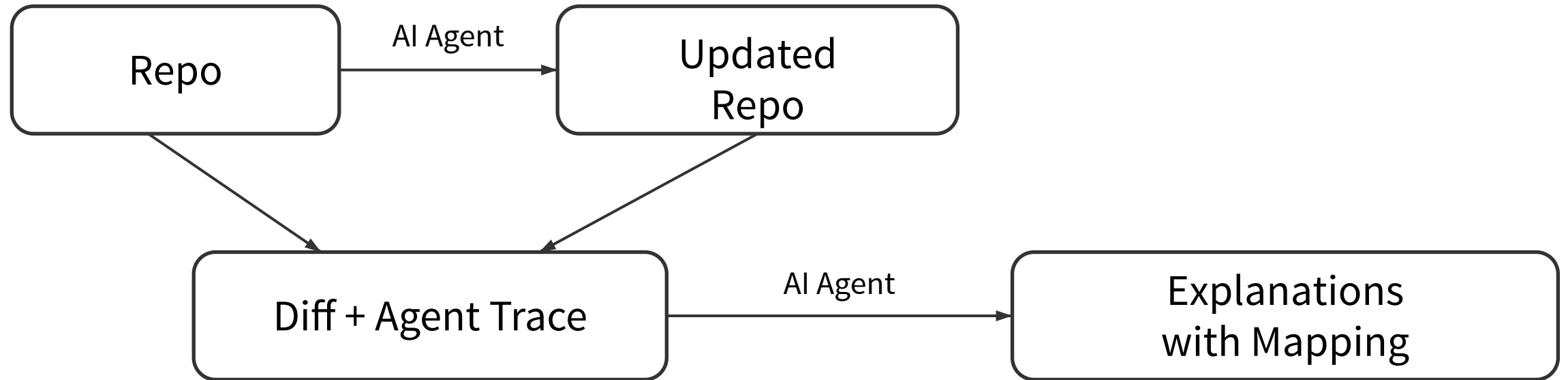
Moves cursor to
the next segment
→

☒ Structured Granularity ☐

The function filters a list of users to find those who are currently active and have been members for more than a year. It then formats each qualifying user's name and email into a string. The result is a list of these strings.

```
1 def process_user_data(users):
2     active_users = [u for u in users if u['status'] == 'active']
3
4     from datetime import datetime, timedelta
5     one_year_ago = datetime.now() - timedelta(days=365)
6     long_term_users = [
7         u for u in active_users if u['join_date'] < one_year_ago
8     ]
9
10    results = []
11    for u in long_term_users:
12        results.append(f"{u['name']} <{u['email']}>")
13    return results
```

INTELLIJ PLUGIN'S HIGH-LEVEL ARCHITECTURE



PLUGIN MODES

AI assistant

Explanation Generator

Code Generation

Explanation Generator

Select code in editor and click Generate to create AI-powered explanations

Explanation Provider: Junie ▾

Detail Level: Medium Detail ▾

Format: Paragraph ▾

Generate Explanation

Add Explanation Comments

Settings

Explanation Generator

AI assistant

Explanation Generator

Code Generation

Junie Code Generation

Enter a natural language prompt to generate or modify code

Prompt:

Generate Code

Add Explanation Comments

Explanation Settings for Code Changes

Configure how changed code will be explained after generation

Explanation Provider: Junie ▾

Detail Level: Medium Detail ▾

Format: Paragraph ▾

Settings

Code Generation

OPENAI MODEL SELECTION

Explanation Settings for Code Changes

Configure how changed code will be explained after generation

Explanation Provider: OpenAI API ▾

Model: gpt-5.4 | \$2.50 ▾

Detail Le

Format:

Settings

Generat

- gpt-5.5 | \$5.00
- gpt-5.4 | \$2.50
- gpt-5.4-mini | \$0.75
- gpt-4.1 | \$2.00
- gpt-4.1-mini | \$0.40
- gpt-4.1-nano | \$0.10
- o3 | \$2.00
- gpt-4o | \$2.50

CUSTOM EXPLANATION STYLE

Explanation Settings for Code Changes

Configure how changed code will be explained after generation

Explanation Provider: Junie ▼

Detail Level: Medium Detail ▼

Format: Paragraph

- Low Detail
- Medium Detail
- High Detail

Settings

Detail level

Explanation Settings for Code Changes

Configure how changed code will be explained after generation

Explanation Provider: Junie ▼

Detail Level: Medium Detail ▼

Format: Paragraph ▼

Paragraph

- Bullet Points

Settings

Format

MAPPING: PARAGRAPH

The image shows a screenshot of an IDE with a Kotlin file named `Main.kt`. The code implements a `binarySearch` function. The AI assistant panel on the right provides a detailed explanation of the code's logic.

```
1 fun main() {  
2  
3 }  
4  
5 fun binarySearch(arr: IntArray, target: Int): Int {  
6     var low = 0  
7     var high = arr.size - 1  
8  
9     while (low <= high) {  
10         val mid = low + (high - low) / 2  
11         when {  
12             arr[mid] == target -> return mid  
13             arr[mid] < target -> low = mid + 1  
14             else -> high = mid - 1  
15         }  
16     }  
17     return -1  
18 }  
19
```

AI assistant
Explanation Generator Code Generation

Code Changes Explanation
Current view: High Detail - Paragraph

Main.kt
Lines 5-19

Binary Search Implementation

The `binarySearch` function efficiently locates a target value in a sorted `IntArray` using two pointers, `low` and `high`, initialized to the array's boundaries. A `while` loop iterates as long as `low <= high`, computing the `mid` index to prevent potential overflow. Inside, a `when` expression compares the element at `mid` with the target to adjust the search range, ultimately returning the target index if found or `-1` if the loop completes without a match.

untitled6 > src > Main.kt > binarySearch 5:1 LF UTF-8 4 spaces

MAPPING: PARAGRAPH

Main.kt

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

```
fun main() {  
  
}  
  
fun binarySearch(arr: IntArray, target: Int): Int { 4 Usages  
    var low = 0  
    var high = arr.size - 1  
  
    while (low <= high) {  
        val mid = low + (high - low) / 2  
        when {  
            arr[mid] == target -> return mid  
            arr[mid] < target -> low = mid + 1  
            else -> high = mid - 1  
        }  
    }  
    return -1  
}
```

AI assistant

Explanation Generator Code Generation

Code Changes Explanation

Current view: High Detail - Paragraph

Main.kt

Lines 5-19

Binary Search Implementation

The binarySearch function efficiently locates a target value in a sorted IntArray using two pointers, low and high, initialized to the array's boundaries. A while loop iterates as long as low <= high, computing the mid index to prevent potential overflow. Inside, a when expression compares the element at mid with the target to adjust the search range, ultimately returning the target index if found or -1 if the loop completes without a match.

untitled6 > src > Main.kt > binarySearch 9:1 LF UTF-8 4 spaces

MAPPING: PARAGRAPH

The image shows a screenshot of an IDE with a Kotlin file named `Main.kt` and an AI assistant panel on the right.

Main.kt Code:

```
1 fun main() {  
2  
3 }  
4  
5 fun binarySearch(arr: IntArray, target: Int): Int { 4 Usages  
6     var low = 0  
7     var high = arr.size - 1  
8  
9     while (low <= high) {  
10         val mid = low + (high - low) / 2  
11         when {  
12             arr[mid] == target -> return mid  
13             arr[mid] < target -> low = mid + 1  
14             else -> high = mid - 1  
15         }  
16     }  
17     return -1  
18 }  
19
```

AI assistant

Explanation Generator Code Generation

Code Changes Explanation

Current view: High Detail - Paragraph

Main.kt

Lines 5-19

Binary Search Implementation

The `binarySearch` function efficiently locates a target value in a sorted `IntArray` using two pointers, `low` and `high`, initialized to the array's boundaries. A `while` loop iterates as long as `low <= high`, computing the `mid` index to prevent potential overflow. Inside, a `when` expression compares the element at `mid` with the target to adjust the search range, ultimately returning the target index if found or `-1` if the loop completes without a match.

untitled6 > src > Main.kt > binarySearch 11:1 LF UTF-8 4 spaces 5.6 / 13

MAPPING: BULLET POINTS

Main.kt

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

```
1 fun main() {  
2  
3 }  
4  
5 fun binarySearch(arr: IntArray, target: Int): Int { 4 Usages  
6     var low = 0  
7     var high = arr.size - 1  
8  
9     while (low <= high) {  
10         val mid = low + (high - low) / 2  
11         when {  
12             arr[mid] == target -> return mid  
13             arr[mid] < target -> low = mid + 1  
14             else -> high = mid - 1  
15         }  
16     }  
17     return -1  
18 }  
19
```

AI assistant

Explanation Generator Code Generation

Code Changes Explanation

Current view: High Detail - Bullet Points

Main.kt

Lines 5-19

Binary Search Implementation

- Implements the standard binary search algorithm in Kotlin.
- Initializes low and high pointers representing the search space boundaries.
- Enters a while loop as long as low <= high.
 - Computes the mid index using low + (high - low) / 2.
 - Utilizes a when expression to handle comparisons.
 - Returns mid if found, shifts low if target is larger, or shifts high if target is smaller.
- Returns -1 if the target is not found after the loop finishes.

untitled6 > src > Main.kt > binarySearch 5:1 LF UTF-8 4 spaces

MAPPING: BULLET POINTS

Main.kt

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

```
1 fun main() {
2
3 }
4
5 fun binarySearch(arr: IntArray, target: Int): Int { 4 Usages
6     var low = 0
7     var high = arr.size - 1
8
9     while (low <= high) {
10         val mid = low + (high - low) / 2
11         when {
12             arr[mid] == target -> return mid
13             arr[mid] < target -> low = mid + 1
14             else -> high = mid - 1
15         }
16     }
17     return -1
18 }
19
```

AI assistant

Explanation Generator Code Generation

Code Changes Explanation

Current view: High Detail - Bullet Points

Main.kt

Lines 5-19

Binary Search Implementation

- Implements the standard binary search algorithm in Kotlin.
- Initializes low and high pointers representing the search space boundaries.
- Enters a while loop as long as low <= high.
 - Computes the mid index using low + (high - low) / 2.
 - Utilizes a when expression to handle comparisons.
 - Returns mid if found, shifts low if target is larger, or shifts high if target is smaller.
- Returns -1 if the target is not found after the loop finishes.

untitled6 > src > Main.kt > binarySearch 6:1 LF UTF-8 4 spaces 5.8 / 13

MAPPING: BULLET POINTS

Main.kt

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

```
fun main() {  
  
}  
  
fun binarySearch(arr: IntArray, target: Int): Int {  
    var low = 0  
    var high = arr.size - 1  
  
    while (low <= high) {  
        val mid = low + (high - low) / 2  
        when {  
            arr[mid] == target -> return mid  
            arr[mid] < target -> low = mid + 1  
            else -> high = mid - 1  
        }  
    }  
    return -1  
}
```

4 Usages

AI assistant

Explanation GeneratorCode Generation

Code Changes Explanation

Current view: High Detail - Bullet Points

Main.kt

Lines 5-19

Binary Search Implementation

- Implements the standard binary search algorithm in Kotlin.
- Initializes low and high pointers representing the search space boundaries.
- Enters a while loop as long as low <= high.
 - Computes the mid index using low + (high - low) / 2.
 - Utilizes a when expression to handle comparisons.
 - Returns mid if found, shifts low if target is larger, or shifts high if target is smaller.
- Returns -1 if the target is not found after the loop finishes.

untitled6 > src > Main.kt > binarySearch9:1 L F UTF-8 4 spaces

RESEARCH HYPOTHESES

H1. The system can generate explanations that are close to human-written explanations in clarity, structure, and grounding in the actual code.

H2. Adding grounded explanations to the repository can improve the coding agent's repair performance without introducing regressions.

LABML.AI REFERENCE DATASET

The dataset contains educational machine-learning code with human-written explanations already mapped to the code regions they describe.

CODE FRAGMENT

```
for t in range(seq_len):  
    i, f, g, o = gates[t].chunk(4)  
    c = f * c + i * g  
    h = o * tanh(c)
```

mapped
->

HUMAN EXPLANATION

Iterates through the sequence one timestep at a time.

Splits the recurrent layer output into LSTM gates.

Updates cell and hidden state using gated memory.

LLM-AS-A-JUDGE EVALUATION

The judge compares generated explanations with the human-written reference. The evaluator model is **OpenAI o3**.

Compare the generated explanation components against the reference components.

Evaluate whether the generated explanation:

- covers the same code behavior
- is correct and clear
- maps to the same code regions

Return a score from 0.0 to 1.0.

LLM-AS-A-JUDGE RESULTS

Provider	Method	Price / 1M input tokens	Mean	Median	Min	Max	Avg. generated components
OpenAI	gpt-4o-mini	\$0.15	0.416	0.500	0.250	0.550	10.2
	gpt-4.1-mini	\$0.40	0.696	0.780	0.500	0.830	19.6
	gpt-4.1	\$2.00	0.786	0.800	0.650	0.850	16.6
Junie	Junie (gemini-3.1-flash-lite)	-	0.752	0.780	0.650	0.800	8.4

H1 confirmed: generated explanations are close to the human-written reference.

Reference explanations contain 25.4 components on average.

SWE-BENCH VERIFIED MINI DATASET

- 50 real bug-fix tasks from Django and Sphinx
- **Input:** starting commit, GitHub issue text
- **Correctness check:** existing tests, test patch
- Is used for code generation systems evaluation

EVALUATION PLAN

Explanations: Junie gemini-3.1-flash-lite-preview

Code generation: Junie gemini-3-flash-preview

1. Run system to compose code explanations on files from gold patch and 10 random files
2. Insert explanations as comments
3. Run each task twice: **baseline repo** and **repo with inserted explanations**
4. Check correctness with the test patch and existing regression tests

SWE-BENCH VERIFIED MINI RESULTS

Dataset	Tasks	Regular solved	Explained solved	Regular rate	Explained rate
Django	25	20	20	80%	80%
Sphinx	25	12	13	48%	52%
Combined	50	32	33	64%	66%

H2 confirmed: explanation-enriched repositories solved one additional task with no observed regression.

CONTRIBUTIONS

1. **Plugin:** implemented a JetBrains IntelliJ plugin for code generation with mapped explanations.
2. **Evaluation:** composed an evaluation approach based on two datasets: labml.ai and SWE-bench Verified Mini.
3. **H1 confirmed:** generated explanations are close to human-written references, with the best mean score reaching **78.6%**.
4. **H2 confirmed:** explanation-enriched repositories achieved **66%** accuracy on SWE-bench Verified Mini.



GitHub
[yule44ka/explainable-ai-plugin](https://github.com/yule44ka/explainable-ai-plugin)